Lab 12
File System and Implementation

Lab Topics

- Lab 1: Symbolic and hard Link (linux/Windows)
- Lab 2: Linux's File Type: Named Pipe (linux only)
- Lab 3: Reading and Writing Files (linux only)

Symbolic and hard Link

- Experiment:
  - Use the concept of general graph directory structure to create the link file by using *ln* command on Linux and mklink on Windows.
  - Create hard/soft links to a source. After delete the source file, what's happen with the symbolic and hard link files?
  - Create level 1 link to source and create level 2 link from level 1. See what happen if you delete source or delete level 1.
  - Create a link to directory or folder. See if you can create link to folder. Moreover, if you could, create a link that create a loop back to upper-level folder. Then try ls –R and dir /s to list the folder recursively and see what happen.
  - Can you create a link on drive D: to source on drive C:?.
- Question:
  - Write a report to show the experimental steps with Linux and windows commands and make the description what you do, discuss the results (step by step), and answer the questions.

Linux's File Type: Named Pipe

- Experiment: (Linux only)
  - Use *mkfifo* comannd to create pipe file by following sample:
    - *mkfifo* pipeFile's name (It can be any name)
  - Use ls command to list the file, and see what you get:
    - *ls –la* pipeFile
  - Use two terminals to access the same pipe file by following instruction:
    - First terminal: type "cat > pipeFile" at the command prompt
    - Second terminal: type "cat pipeFile" at the command prompt
  - At the first terminal, type "hello linux". Then see what you get in the second terminal. You can repeat this action by typing anything at first terminal and see what you get. When you have no more input, try ^D which indicates end of file.
- Question:
  - Write a report that what is the concept of this file type and why this experiment behaves this way.

Reading and Writing Files

- Experiment:
  - Understand the system process for Reading and Writing a file by following the step below:
    - First, create a simple text file by "*echo hello > foo"*
    - Next, using special command to trace the system calls activity, called *strace* command. This case "*strace cat foo*"
    - You can see the output be like this:

```
…
read(3, "hello\n", 32768)           = 6
write(1, "hello\n", 6hello)             = 6
read(3, "", 32768)                  = 0
close(3)                        = 0
…
```

- Question:
  - Write a report that show the explanation of the output (focus four lines). What does the numbers { 3, 1, 6, 32768 } represent?